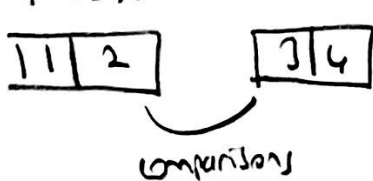
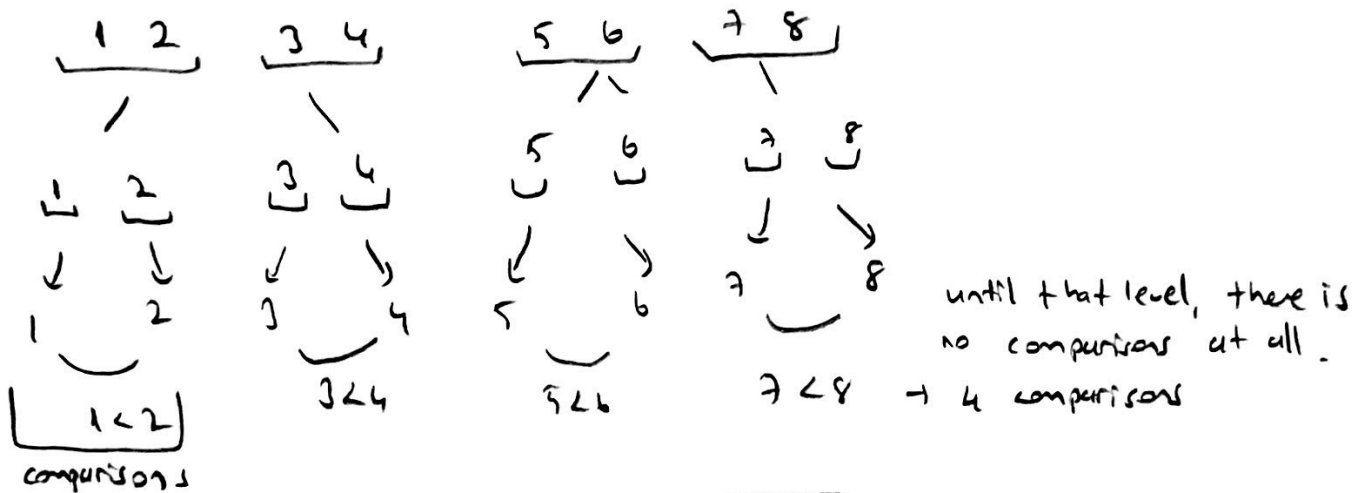


1) Merge Sort

a) Maximum Comparison

Mergesort time complexity $O(n \log n)$. It divides array 2 parts until get subsets which have 1 element. Then combine those subsets recursively.

1 2 3 4 5 6 7 8 → Let say this is our array.



this part has 2 comparison
If my list look like

[5, 7] [6, 8] → I have 3 comparisons

1 2
3 4

1) 1 < 2 ? ✓

2) 2 < 3 ?

1 2 3 4

if my list

1 3 2 4 I have 3 comparisons

1) 1 < 2

2) 2 < 3

3) 3 < 4

1 3 2 4 5 7 6 8

1 3 2 4

5 7 6 8 → how many comparisons

1 3 2 4
 ↓
 5 7 6 8) → 1 3 2 4 5 7 6 8

1) 1 < 5
 2) 3 < 5
 3) 2 < 5
 4) 4 < 5

4 comparisons only, but if my list look like that

1 3 5 7
 2 4 6 8)

1 2 3 4 5 6 7 8

1) 1 < 2
 2) 2 < 3
 3) 3 < 4
 4) 4 < 5
 5) 5 < 6
 6) 6 < 7
 7) 7 < 8

7 comparison → so I update my list first position.

1 3 5 7 2 4 6 8
 ↑ ↑ → I mentioned before

1 5 3 7 2 6 4 8

1, 5 3, 7
 ^ ^
 1 5 3 7
))

2, 6 4, 8
 ^ ^
 2 6 4 8
))

→ 4 comparison

1 5

3 7

2 6

4 8

1) 1 < 3
 2) 3 < 5
 3) 5 < 7

1 3 5 7

2 4 6 8

→ 6 comparison

1 2 3 4 5 6 7 8

1) 1 < 2
 2) 2 < 3
 3) 3 < 4
 4) 4 < 5

5) 5 < 6
 6) 6 < 7
 7) 7 < 8

→ 7

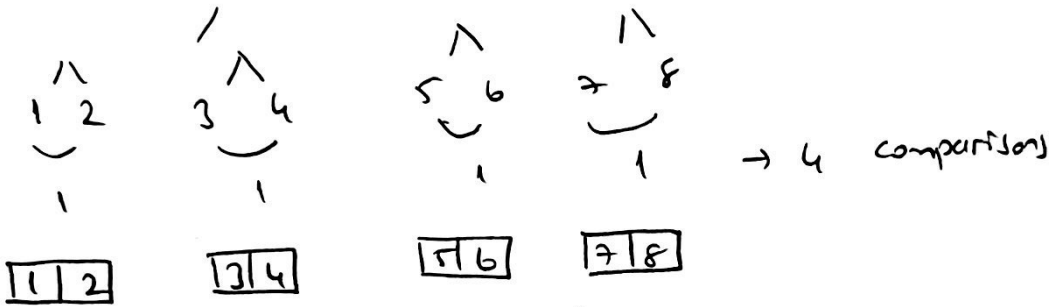
In total 4 + 6 + 7 = 17 comparisons

1)

b) Min comparisons

If I have sorted any that would be minimum number of comparisons in merge sort.

1 2 3 4 5 6 7 8
⋮



1) $1 < 3$
2) $2 < 4$
2 comparisons

[1 | 2 | 3 | 4]

2 comp. → 4 comparisons

[1 | 2 | 3 | 4]

[5 | 6 | 7 | 8]

list greater than left one, this is only comparison.

1) $1 < 5$
2) $2 < 5$
3) $3 < 5$
4) $4 < 5$

1 2 3 4 5 6 7 8
└──────────┘
already listed

4 comparisons

→ 4
+

$4 + 4 + 4 = 12$ comparisons

2.

Find index $i \rightarrow AC[i] = 0$

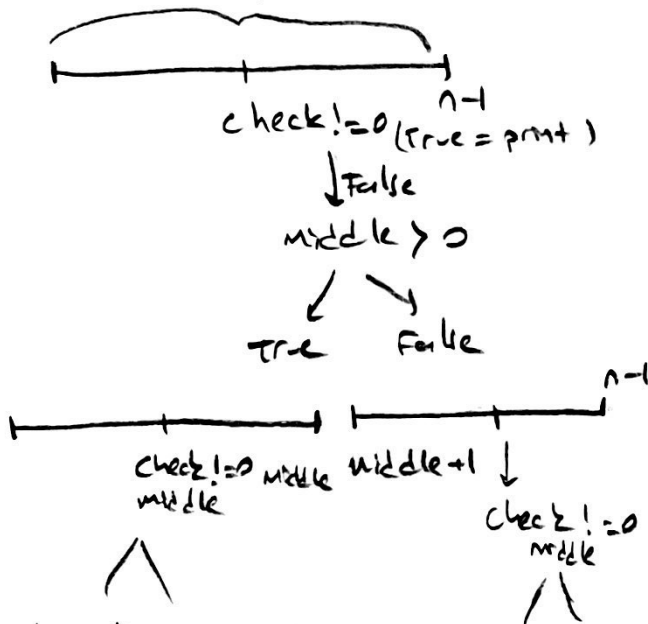
Apply Binary Search Approach

Array has ascending order.

$-7 -5 -1 0 12 24 \dots$ $\rightarrow$ like that

In binary search check the middle element, recursively divide array into two parts.

- If middle one is zero print that index
- If middle greater than zero, that means 0 element is in left part so new middle will be $\frac{0 + \text{array}[\text{middle}]}{2}$
- If middle smaller than zero, that means 0 element is in right part so new middle will be $\frac{\text{array}[\text{middle} + 1] + n - 1}{2}$



Pseudocode

```

int find_zero(int array[], int first, int last) {
    int middle =  $\frac{\text{first} + \text{last}}{2}$ 
    if (arr[middle] == 0) {
        return middle;
    }
    else if (arr[middle] > 0) {
        return find_zero(array[], first, (middle - 1));
    }
    else if (arr[middle] < 0) {
        return find_zero(array[], (middle + 1), last);
    }
}

```

Time complexity

$$T(n) = T\left(\frac{n}{2}\right) + c$$

Each recursion divide problem into 2 pieces.

$$O(\log n)$$

$$\text{Best case } O(1)$$

$$\text{average } O(\log n)$$

Space comp.

Input size n

output $\rightarrow$ constant (1)

recursive search size $\log n$

A hand-drawn diagram of a square box. Inside the box, there are several 'x' marks scattered throughout. In the lower-left area, there is a dot followed by the text '(plane)'. In the lower-right area, there is a handwritten word that appears to be 'tanker' with an arrow pointing towards the bottom center.

$$\text{Distance} = \sqrt{(\text{plane}_x - \text{tanker}_x)^2 + (\text{plane}_y - \text{tanker}_y)^2}$$

Tanker-arr = { (1, 2) , (3, 4) , (5, 6) , ... }

↓ ↓
x coord. y coordinates.

→ Divide the array 2 parts.

Assign d minimum between left and right parts. $d = \min(d_L, d_R)$

→ compare d and d_{new} . If d_{new} is smaller than d , then update d .

Find - closest - take (Take-array) }

calculate left distance d_L

Find minimum of d_L and d_R ($d = \min(d_R, d_L)$)

Ser+

$$f(\text{draw } d) \}$$

Scanned with CamScanner

Time complexity

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log n)$$

↓
divide
2 part

↘ size $n/2$ ↗ sorting

$$a=2 \quad b=2 \quad n'$$

$$2=2'$$

$$O(n \log n \cdot \log n) = O(n \log^2 n)$$

Space complexity

input = n size

output = 1 (constant)

recursive array size $\log n \rightarrow O(\log n)$

4. We try to find miss production. Let's say this product is heavier than the regular

First at all divide equal 3 parts. (If some parts don't equal size, they make it almost equal)

$n \bmod 3 = 0 \rightarrow$ divide 3 parts $\frac{n}{3}$ pieces.

$n \bmod 3 = 1 \rightarrow$ divide 3 parts which sizes are $k+1, k+1, k-1$

$n \bmod 3 = 2 \rightarrow$ divide 3 parts which sizes are $k+1, k+1, k$

Think about all 3 cases. If size of parts equal but weights not equal, then miss product is in heavier part.

Apply divide function required. If size of parts equal and weights equal, then miss product is in the unmeasured part. Apply divide function required for that part.

find-miss-match(array A)

$n = \text{length of array A}$

if $n = 1$

this is the product we are looking for

else if $n = 2$

divide two parts $(n/2) = A, B$

compare A and B

heavier one is the product

else

divide products into 3 parts $(n/3 = \text{ceil}(n/3))$

$C = n - A - B$

compare A and B

if equal, then C has product

find-miss-match(C)

else

if A heavier than B

find-miss-match(A)

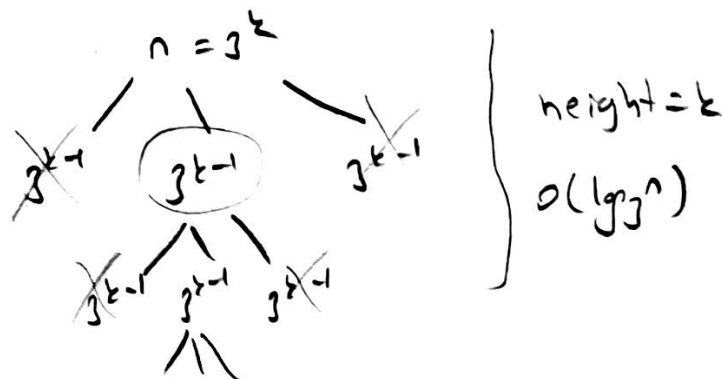
else

find-miss-match(B)

$$T(n) = \begin{cases} T(1) = 0 & n = 1 \\ T(2) = 1 & n = 2 \\ T(n/3) + f(n) & n \geq 3 \end{cases}$$

How many weighings does this require?

$n = 3^k$ (Assume)



Space complexity

input = n size

output = 1

recursive = $\log_3 n$